Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

по дисциплине «Архитектура автоматизированных систем обработки информации и управления» Профиль: «Архитектура программных систем»

Учебные материалы домашнего задания №2 Разработка консольного приложения с применением ООП

Москва – 2026

Оглавление

1 ВВЕДЕНИЕ 3

2 ВАРИАНТЫ ЗАДАНИЙ 3

3 ЧАСТЬ 1. CSV-ФАЙЛЫ С ТЕСТОВЫМИ ДАННЫМИ 6

4 ЧАСТЬ 2. КЛАССЫ-МОДЕЛИ 6

5 ЧАСТЬ 3. КЛАСС DATABASEMANAGER 8

6 ЧАСТЬ 4. КОНСОЛЬНОЕ МЕНЮ 9

7 ЧАСТЬ 5. КЛАСС REPORTBUILDER (FLUENT INTERFACE) 10

8 ЧАСТЬ 6. ТРИ ОБЯЗАТЕЛЬНЫХ ОТЧЁТА 12

9 ЗАДАНИЯ ПО ГРУППАМ 13

10 МЕТАГРАФОВОЕ СИТУАЦИОННОЕ ОПИСАНИЕ 14

10.1 Модель данных 14

10.2 Схема работы класса ReportBuilder 15

11 ТРЕБОВАНИЯ К ОТЧЕТУ 17

12 ТЕКСТ ПРИМЕРА ДЛЯ ПРЕДМЕТНОЙ ОБЛАСТИ СЕМИНАРА 17

12.1 Файл Department.cs 17

12.2 Файл Developer.cs 19

12.3 Файл DatabaseManager.cs 20

12.4 Файл ReportBuilder.cs 26

12.5 Основная программа 30

1 Введение

На семинаре мы работали с таблицами отделов (`dep`) и разработчиков (`dev`) — читали CSV-файлы, загружали данные в SQLite, выполняли запросы. Но данные были доступны только для чтения, а код состоял из отдельных функций.

В этом задании мы создадим полноценное консольное приложение на основе классов для работы с аналогичной парой таблиц вашего варианта. Приложение будет:

- хранить данные в SQLite;
- позволять добавлять, редактировать и удалять записи через консольное меню;
- формировать отчёты с помощью паттерна Fluent Interface.

Все примеры в задании даны для семинарской предметной области (отделы и разработчики). Вы будете реализовывать аналогичную функциональность для предметной области вашего варианта.

В этом домашнем задании мы не используем ключевое слово `interface` и технологию LINQ — они будут изучены позже. Паттерн Fluent Interface — это приём проектирования класса (методы возвращают `this`), он не связан с ключевым словом `interface` в C#.

2 Варианты заданий

Номер варианта — по номеру в списке группы. Структура для всех вариантов для связи двух таблиц один-ко-многим одинаковая:

- Справочная таблица (сторона «один»): идентификатор и название.

- Основная таблица (сторона «много»): идентификатор, внешний ключ на справочник, название и числовой показатель.

Для сравнения — семинарский пример: справочник `dep(dep_id, dep_name)`, основная таблица `dev(dev_id, dep_id, dev_name, dev_commits)`.

№	Справочник	Основная таблица	Числовое поле	Расшифровка
1	Жанры	Книги	`pages`	Количество страниц
2	Киностудии	Фильмы	`budget_mln`	Бюджет фильма (млн долл.)
3	Автомобильные марки	Автомобили	`horsepower`	Мощность двигателя (л.с.)
4	Факультеты	Студенты	`gpa`	Средний балл успеваемости
5	Категории товаров	Товары	`price`	Цена (руб.)
6	Должности	Работники	`salary`	Зарплата (тыс. руб./мес.)
7	Музыкальные альбомы	Песни	`duration_sec`	Длительность трека (сек.)
8	Кухни мира	Рецепты	`cook_min`	Время приготовления (мин.)
9	Страны	Города	`population_k`	Население (тыс. чел.)
10	Лаборатории	Эксперименты	`duration_hours`	Длительность опыта (часов)
11	Производители электроники	Смартфоны	`price`	Цена (руб.)
12	Игровые	Видеоигры	`rating`	Оценка

	платформы			игроков (баллы из 100)
13	Музеи	Экспонаты	`value_k`	Оценочная стоимость (тыс. руб.)
14	Авиакомпании	Рейсы	`distance_km`	Дальность маршрута (км)
15	Футбольные клубы	Футболисты	`goals`	Количество забитых голов
16	Больницы	Врачи	`experience`	Стаж работы (лет)
17	Издательства	Журналы	`circulation_k`	Тираж (тыс. экземпляров)
18	Кафедры	Преподаватели	`publications`	Количество научных публикаций
19	Магазины	Заказы	`amount`	Сумма заказа (руб.)
20	Проекты	Задачи	`hours`	Трудоёмкость (человеко-часов)
21	Микрорайоны	Жилые дома	`floors`	Количество этажей
22	Рестораны	Блюда в меню	`price`	Цена блюда (руб.)
23	Теплицы	Растения	`height_cm`	Высота растения (см)
24	Операторы связи	Тарифы	`price_month`	Абонентская плата

				(руб./мес.)
25	Зоопарки	Животные	`weight_kg`	Масса животного (кг)
26	Школы	Ученики	`avg_grade`	Средний балл успеваемости
27	Серверы	Базы данных	`size_gb`	Объём данных (ГБ)
28	Склады	Партии товаров	`quantity`	Количество единиц (шт.)
29	Автопарки	Транспортные средства	`mileage_km`	Пробег (км)
30	Телеканалы	Телепередачи	`viewers_k`	Аудитория (тыс. зрителей)

3 Часть 1. CSV-файлы с тестовыми данными

Создайте два CSV-файла для вашего варианта. Формат — как на семинаре: разделитель — точка с запятой `;`, первая строка — заголовки.

Минимальный объём данных:

- справочник — не менее 4 записей;

- основная таблица — не менее 12 записей (по 2–4 записи на каждую категорию — чтобы отчёты с группировкой выглядели содержательно).

4 Часть 2. Классы-модели

Создайте по одному классу для каждой таблицы (аналоги `Department` и `Developer`).

Требования к каждому классу:

- Для каждого поля таблицы нужно использовать не переменные а свойства (properties).

- У числового поля — валидация в set-аксессоре: если значение отрицательное, генерируется `ArgumentException`.

- Два конструктора: полный (со всеми параметрами) и конструктор по умолчанию, вызывающий полный через цепочку `this(...)`.

- Переопределение `ToString()` для удобного вывода.

- XML-комментарии для класса и его публичных членов.

```
/// <summary>
/// Разработчик
/// </summary>
class Developer
{
    public int Id { get; set; }
    public int DepartmentId { get; set; }
    public string Name { get; set; }

    private int _commits;
    /// <summary>
    /// Количество коммитов (не может быть отрицательным)
    /// </summary>
    public int Commits
    {
        get => _commits;
        set
        {
            if (value < 0)
                throw new ArgumentException(
                    "Количество коммитов не может быть отрицательным");
            _commits = value;
        }
    }

    /// <summary>
    /// Конструктор с параметрами
    /// </summary>
    public Developer(int id, int departmentId, string name, int commits)
    {
        Id = id;
        DepartmentId = departmentId;
        Name = name;
        Commits = commits; // валидация сработает здесь
    }

    /// <summary>
    /// Конструктор по умолчанию
    /// </summary>
    public Developer() : this(0, 0, "", 0) { }

    public override string ToString()
        => $"[{Id}] {Name}, отдел #{DepartmentId}, коммитов: {Commits}";
}
```

Подсказка. Класс для справочной таблицы устроен проще — у него всего два свойства (`Id` и `Name`), но `ToString()` и конструкторы всё равно нужны.

5 Часть 3. Класс DatabaseManager

Создайте класс `DatabaseManager`, который инкапсулирует всё взаимодействие с SQLite.

Что должен уметь `DatabaseManager`:

Метод	Описание
Конструктор	Принимает путь к файлу БД. Если файла нет — создаёт БД и таблицы
`ImportFromCsv`	Читает два CSV-файла и заполняет таблицы
`GetAllDepartments`	Возвращает `List<Department>`
`GetAllDevelopers`	Возвращает `List<Developer>`
`AddDeveloper`	Вставляет новую запись
`UpdateDeveloper`	Обновляет существующую запись по Id
`DeleteDeveloper`	Удаляет запись по Id

Для отчётов (часть 5) `DatabaseManager` должен также предоставить метод, который принимает строку SQL-запроса и возвращает результат, пригодный для форматирования. Этот метод будет использоваться классом `ReportBuilder`. Вы можете использовать любой подход, который был рассмотрен на семинаре, например подход с использованием записей:

```
/// <summary>
/// Одна строка CSV-файла. Хранит значения полей в порядке столбцов.
/// record даёт неизменяемость и готовый ToString() для отладки.
/// </summary>
record CsvRow(string[] Fields);

/// <summary>
/// CSV-таблица целиком: заголовки (имена столбцов) + список строк.
/// </summary>
record CsvTable(string[] Headers, List<CsvRow> Rows);
```

Подсказка. Все обращения к `SqliteConnection` должны использовать конструкцию `using`:

```
public List<Developer> GetAllDevelopers()
{
    var result = new List<Developer>();
    using var connection = new SqliteConnection(_connectionString);
    connection.Open();
    var cmd = connection.CreateCommand();
    cmd.CommandText = "SELECT * FROM dev ORDER BY dev_id";
    using var reader = cmd.ExecuteReader();
    while (reader.Read())
    {
        result.Add(new Developer(
            reader.GetInt32(0),
            reader.GetInt32(1),
            reader.GetString(2),
            reader.GetInt32(3)
        ));
    }
    return result;
}
```

Подсказка по архитектуре приложения. Храните строку подключения (`_connectionString`) как закрытое поле класса — она используется во всех методах.

6 Часть 4. Консольное меню

Реализуйте интерактивное меню в цикле. Программа работает, пока пользователь не выберет «Выход».

```
=== Управление данными ===
 1 — Показать все отделы
 2 — Показать всех разработчиков
 3 — Добавить разработчика
 4 — Редактировать разработчика
 5 — Удалить разработчика
 6 — Отчёты
 0 — Выход
Ваш выбор:
```

Требования:

- При добавлении — сначала показать список записей справочника, чтобы пользователь мог выбрать внешний ключ.

- При редактировании — показать текущие значения полей, затем запросить новые. Если пользователь нажимает Enter без ввода — поле не изменяется.

- При ошибках ввода — вывести понятное сообщение и не завершать программу (достаточно простого `try/catch` вокруг операции).

Подсказка. Редактируются только записи основной таблицы (сторона «много»). Справочная таблица — только для просмотра.

Подсказка по вводу числовых значений. Используйте `int.TryParse` для безопасного ввода:

```
Console.Write("Количество коммитов: ");
if (!int.TryParse(Console.ReadLine(), out int commits))
{
    Console.WriteLine("Ошибка: введите целое число");
    continue;  // или return, в зависимости от структуры кода
}
```

7 Часть 5. Класс ReportBuilder (Fluent Interface)

Создайте класс `ReportBuilder`, реализующий паттерн Fluent Interface для формирования и вывода отчётов.

`ReportBuilder` получает ссылку на `DatabaseManager` через конструктор и использует его для выполнения SQL-запросов:

```
class ReportBuilder
{
    private DatabaseManager _db;

    // ... внутренние поля для настроек отчёта ...

    /// <summary>
    /// Конструктор принимает DatabaseManager для доступа к данным
    /// </summary>
    public ReportBuilder(DatabaseManager db)
    {
        _db = db;
    }

    // ... промежуточные и терминальные методы ...
}
```

Обратите внимание: класс `ReportBuilder` напрямую зависит от класса `DatabaseManager` — это жёсткая связь (tight coupling). На одном из следующих занятий мы обсудим, как сделать связь между классами более гибкой.

Промежуточные методы (возвращают `this`). Каждый промежуточный метод сохраняет часть настроек отчёта во внутреннем поле и возвращает `this`, что позволяет записывать вызовы в цепочку:

Метод	**Что настраивает**
`Query(string sql)`	SQL-запрос, который нужно выполнить
`Title(string text)`	Заголовок отчёта
`Header(params string[] columns)`	Названия колонок для отображения
`ColumnWidths(params int[] widths)`	Ширина каждой колонки в символах

Терминальные методы (завершают цепочку)

Метод	**Что делает**
`Build()`	Выполняет SQL через `_db`, форматирует результат с помощью `StringBuilder`, возвращает строку
`Print()`	Вызывает `Build()` и выводит результат в консоль

Пример использования:

```
var db = new DatabaseManager("mydata.db");
        // ...

        // Создаём отчёт — цепочка вызовов
        new ReportBuilder(db)
    .Query(@"SELECT d.dev_name, dep.dep_name, d.dev_commits
             FROM dev d
             JOIN dep ON d.dep_id = dep.dep_id
             ORDER BY d.dev_name")
    .Title("Разработчики по отделам")
    .Header("Имя", "Отдел", "Коммиты")
    .ColumnWidths(20, 15, 10)
    .Print();
```

Результат в консоли:

```
=== Разработчики по отделам ===
Имя                    Отдел            Коммиты
──────────────────────────────────────────────────
Иванов                 Backend          120
Козлова                Frontend         110
Новиков                QA               45
Петров                 Backend          95
Сидоров                Frontend         80
```

Подсказка по реализации. Внутри класса заведите поля:

```
private string _sql = "";
private string _title = "";
private string[] _headers = Array.Empty<string>();
private int[] _widths = Array.Empty<int>();
```

Каждый промежуточный метод заполняет своё поле и делает `return this`. Терминальный метод `Build()` передаёт `_sql` в `DatabaseManager`, читает результат и форматирует его через `StringBuilder`.

Подсказка по `StringBuilder`. Для форматирования строки фиксированной ширины используйте строковую интерполяцию с выравниванием:

```
// Вывод строки шириной 20 символов с выравниванием влево
sb.Append($"{value,-20}");
```

8 Часть 6. Три обязательных отчёта

С помощью `ReportBuilder` реализуйте три отчёта. SQL-запросы пишутся как обычные строки (не нужно строить SQL динамически — просто напишите готовый запрос для каждого отчёта).

Отчёт 1. Полный список записей основной таблицы с названиями из справочника (используется `JOIN`, сортировка по названию).

Для семинарской области: список разработчиков с названиями отделов, отсортированный по имени.

Отчёт 2. Количество записей в каждой категории (используется `GROUP BY` с `COUNT(*)`).

Для семинарской области: количество разработчиков в каждом отделе. Пример запроса SQL:

```
SELECT dep.dep_name, COUNT(*) AS cnt
FROM dev
JOIN dep ON dev.dep_id = dep.dep_id
GROUP BY dep.dep_name
```

Отчёт 3. Среднее значение числового поля по категориям (используется `GROUP BY` с `AVG(...)`), с сортировкой.

Для семинарской области: среднее количество коммитов по отделам.

Доступ к отчётам реализуется через пункт «Отчёты» консольного меню.

9 Задания по группам

Помимо основных заданий, каждая группа выполняет одно дополнительное задание:

Группа	Дополнительное задание
1 (А)	Добавить в `ReportBuilder` промежуточный метод `Numbered()`. Если он вызван в цепочке, терминальный метод добавляет в вывод нумерацию строк (столбец «№» слева).
2 (Б)	Добавить в `ReportBuilder` терминальный метод `SaveToFile(string path)`, который сохраняет отформатированный отчёт в текстовый файл (вместо вывода в консоль).
3 (В)	Добавить в `ReportBuilder` промежуточный метод `Footer(string label)`. Если он вызван, в конце отчёта добавляется итоговая строка с текстом (например, *«Всего

	записей: 15»*).
4 (Г)	Добавить в консольное меню пункт «Фильтр по категории» — пользователь выбирает запись из справочника, программа выводит только записи основной таблицы, принадлежащие этой категории.

Пример для группы А:

```
new ReportBuilder(db)
    .Query("SELECT ... ")
    .Title("Разработчики")
    .Header("Имя", "Отдел", "Коммиты")
    .ColumnWidths(5, 20, 15, 10) // +5 для столбца "№"
    .Numbered()  // ← дополнительный метод
    .Print();
```

10 Метаграфовое ситуационное описание

Разберем сложные фрагменты проекта с помощью метаграфового ситуационного описания.

Для рисования схем Вы можете использовать любой графический редактор.

10.1 Модель данных

Это два класса, связанные внешним ключом — здесь схема помогает увидеть структуру до написания кода.

Изобразите ваши два класса-модели как метавершины (прямоугольники). Внутри каждой «коробки» — поля в виде строк с пометками типа и ограничений. Соедините коробки стрелкой, показывающей связь «один ко многим» через внешний ключ. Пример для семинарской предметной области (ваш вариант — аналогично):

```
╔═══════════════════════╗                 ╔═══════════════════════════╗
║ Department            ╕                 ║ Developer                 ╕
│ ─────────────────────│                 │ ──────────────────────────│
│ Id   : int  [PK]      │──1──────M──►    │ Id           : int  [PK]  │
│ Name : string         │                 │ DepartmentId : int  [FK →Dep]│
╘═══════════════════════╛                 │ Name         : string     │
                                          │ Commits      : int  [≥ 0] │
                                          ╘═══════════════════════════╛
```

На схеме должно быть видно:

- имена полей и их типы;
- какое поле является первичным ключом (PK);
- какое поле является внешним ключом (FK) и на что оно ссылается;
- для числового поля — ограничение (например, ≥ 0).

10.2 Схема работы класса ReportBuilder

Цепочка вызовов .Query(...).Title(...).Header(...).ColumnWidths(...).Print() — это процесс: каждый метод добавляет новую «деталь» к внутреннему состоянию объекта ReportBuilder. Нарисуйте этот процесс как последовательность состояний, где стрелки — это вызовы методов, а прямоугольники — состояния объекта после каждого вызова.

Шаблон схемы (заполните для одного из ваших трёх отчётов):

```
┌──────────────┐          ┌──────────────────────┐
│ ReportBuilder│          │ ReportBuilder        │
│ (начало)     │─Query─►  │ _sql = "SELECT ..."  │
│              │          └──────────────────────┘
│ _sql = ""    │                     │
│ _title = ""  │                .Title(...)
│ _headers = []│                     │
│ _widths = [] │                     ▼
└──────────────┘
                          ┌──────────────────────┐
                          │ ReportBuilder        │
                          │ _sql   = "SELECT ..." │
                          │ _title = "Название отч." │
                          └──────────────────────┘
                                      │
                                 .Header(...)
                                      │
                                      ▼
                          ┌──────────────────────┐
                          │ ReportBuilder        │
                          │ _sql     = "SELECT ..." │
                          │ _title   = "..."      │
                          │ _headers = ["Кол.1","…"] │
                          └──────────────────────┘
                                      │
                            .ColumnWidths(...)
                                      │
                                      ▼
                          ┌──────────────────────┐
                          │ ReportBuilder        │
                          │ _sql     = "SELECT ..." │
                          │ _title   = "..."      │
                          │ _headers = [...]      │
                          │ _widths  = [20, 15, 10] │
                          └──────────────────────┘
                                      │
                                  .Print()
                                      │
                                      ▼
                            ┌──────────────┐
                            │ Вывод в консоль │
                            │ (отчёт готов)   │
                            └──────────────┘
```

Зачем это нужно?

Рисуя схему А, вы проверяете, что правильно понимаете связь между таблицами — до того, как напишете SQL.

Рисуя схему Б, вы понимаете механизм Fluent Interface: метод возвращает this именно потому, что передаёт «обогащённое» состояние объекта следующему

методу в цепочке. Если схема нарисована — реализация ReportBuilder пишется почти автоматически.

Дополнительное задание:

С использованием plantuml:

- Реализуйте схему А в форме нотации Чена (https://plantuml.com/ru/er-diagram) и в форме Entity Relationship Diagram (https://plantuml.com/ru/ie-diagram)
- Реализуйте схему Б в форме Activity Diagramm UML - https://plantuml.com/ru/activity-diagram-beta

11 Требования к отчету

Для сдачи домашнего задания необходимо продемонстрировать работоспособность программы, ответить на вопросы и предоставить отчет, содержащий текст программы и результаты ее выполнения.

12 Текст примера для предметной области семинара

12.1 Файл Department.cs

```
/// <summary>
/// Отдел (справочная таблица, сторона «один»)
/// </summary>
class Department
{
    /// <summary>Идентификатор отдела</summary>
    public int Id { get; set; }

    /// <summary>Название отдела</summary>
    public string Name { get; set; }

    /// <summary>Конструктор с параметрами</summary>
    public Department(int id, string name)
    {
        Id = id;
        Name = name;
    }

    /// <summary>Конструктор по умолчанию</summary>
```

```
    public Department() : this(0, "") { }

    public override string ToString() => $"[{Id}] {Name}";
}
```

12.2 Файл Developer.cs

```
/// <summary>
/// Разработчик (основная таблица, сторона «много»)
/// </summary>
class Developer
{
    /// <summary>Идентификатор разработчика</summary>
    public int Id { get; set; }

    /// <summary>Идентификатор отдела (внешний ключ)</summary>
    public int DepartmentId { get; set; }

    /// <summary>Имя разработчика</summary>
    public string Name { get; set; }

    private int _commits;
    /// <summary>
    /// Количество коммитов (не может быть отрицательным)
    /// </summary>
    public int Commits
    {
        get => _commits;
        set
        {
            if (value < 0)
                throw new ArgumentException(
                    "Количество коммитов не может быть отрицательным");
            _commits = value;
        }
    }

    /// <summary>Конструктор с параметрами</summary>
    public Developer(int id, int departmentId, string name, int commits)
    {
        Id = id;
        DepartmentId = departmentId;
        Name = name;
        Commits = commits;
    }

    /// <summary>Конструктор по умолчанию</summary>
    public Developer() : this(0, 0, "", 0) { }

    public override string ToString()
        => $"[{Id}] {Name}, отдел #{DepartmentId}, коммитов: {Commits}";
}
```

12.3 Файл DatabaseManager.cs

```
using Microsoft.Data.Sqlite;

/// <summary>
/// Управление базой данных SQLite.
/// Инкапсулирует все операции с БД: создание таблиц,
/// импорт CSV, CRUD-операции, выполнение запросов для отчётов.
/// </summary>
class DatabaseManager
{
    private string _connectionString;

    /// <summary>
    /// Конструктор. Принимает путь к файлу БД.
    /// </summary>
    public DatabaseManager(string dbPath)
    {
        _connectionString = $"Data Source={dbPath}";
    }

    // ──────────── Инициализация ────────────

    /// <summary>
    /// Создаёт таблицы (если не существуют) и загружает CSV при первом
запуске
    /// </summary>
    public void InitializeDatabase(string depCsvPath, string devCsvPath)
    {
        CreateTables();

        // Импорт только если таблицы пусты
        if (GetAllDepartments().Count == 0 && File.Exists(depCsvPath))
        {
            ImportDepartmentsFromCsv(depCsvPath);
            Console.WriteLine($"[OK] Загружены отделы из {depCsvPath}");
        }

        if (GetAllDevelopers().Count == 0 && File.Exists(devCsvPath))
        {
            ImportDevelopersFromCsv(devCsvPath);
            Console.WriteLine($"[OK] Загружены разработчики из
{devCsvPath}");
        }
    }

    /// <summary>Создание таблиц</summary>
    private void CreateTables()
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
        CREATE TABLE IF NOT EXISTS dep (
```

```
        dep_id   INTEGER PRIMARY KEY AUTOINCREMENT,
        dep_name TEXT NOT NULL
    );
    CREATE TABLE IF NOT EXISTS dev (
        dev_id      INTEGER PRIMARY KEY AUTOINCREMENT,
        dep_id      INTEGER NOT NULL,
        dev_name    TEXT    NOT NULL,
        dev_commits INTEGER NOT NULL,
        FOREIGN KEY (dep_id) REFERENCES dep(dep_id)
    );";
    cmd.ExecuteNonQuery();
}

/// <summary>Импорт отделов из CSV</summary>
private void ImportDepartmentsFromCsv(string path)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();

    string[] lines = File.ReadAllLines(path);
    for (int i = 1; i < lines.Length; i++)
    {
        string[] parts = lines[i].Split(';');
        if (parts.Length < 2) continue;

        var cmd = conn.CreateCommand();
        cmd.CommandText =
            "INSERT INTO dep (dep_id, dep_name) VALUES (@id, @name)";
        cmd.Parameters.AddWithValue("@id", int.Parse(parts[0]));
        cmd.Parameters.AddWithValue("@name", parts[1]);
        cmd.ExecuteNonQuery();
    }
}

/// <summary>Импорт разработчиков из CSV</summary>
private void ImportDevelopersFromCsv(string path)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();

    string[] lines = File.ReadAllLines(path);
    for (int i = 1; i < lines.Length; i++)
    {
        string[] parts = lines[i].Split(';');
        if (parts.Length < 4) continue;

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
        INSERT INTO dev (dev_id, dep_id, dev_name, dev_commits)
        VALUES (@id, @depId, @name, @commits)";
        cmd.Parameters.AddWithValue("@id", int.Parse(parts[0]));
        cmd.Parameters.AddWithValue("@depId", int.Parse(parts[1]));
        cmd.Parameters.AddWithValue("@name", parts[2]);
        cmd.Parameters.AddWithValue("@commits", int.Parse(parts[3]));
        cmd.ExecuteNonQuery();
    }
```

```
    }

    // ──────────── Чтение данных ────────────

    /// <summary>Получить все отделы</summary>
    public List<Department> GetAllDepartments()
    {
        var result = new List<Department>();
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT dep_id, dep_name FROM dep ORDER BY
dep_id";
        using var reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            result.Add(new Department(
                reader.GetInt32(0),
                reader.GetString(1)));
        }
        return result;
    }

    /// <summary>Получить всех разработчиков</summary>
    public List<Developer> GetAllDevelopers()
    {
        var result = new List<Developer>();
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText =
            "SELECT dev_id, dep_id, dev_name, dev_commits FROM dev ORDER BY
dev_id";
        using var reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            result.Add(new Developer(
                reader.GetInt32(0),
                reader.GetInt32(1),
                reader.GetString(2),
                reader.GetInt32(3)));
        }
        return result;
    }

    /// <summary>Получить разработчика по Id</summary>
    public Developer GetDeveloperById(int id)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
```

```
        cmd.CommandText =
            "SELECT dev_id, dep_id, dev_name, dev_commits FROM dev WHERE
dev_id = @id";
        cmd.Parameters.AddWithValue("@id", id);
        using var reader = cmd.ExecuteReader();

        if (reader.Read())
        {
            return new Developer(
                reader.GetInt32(0), reader.GetInt32(1),
                reader.GetString(2), reader.GetInt32(3));
        }
        return null;
    }

    // ──────────── Изменение данных ────────────

    /// <summary>Добавить разработчика (Id генерируется
автоматически)</summary>
    public void AddDeveloper(Developer dev)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
        INSERT INTO dev (dep_id, dev_name, dev_commits)
        VALUES (@depId, @name, @commits)";
        cmd.Parameters.AddWithValue("@depId", dev.DepartmentId);
        cmd.Parameters.AddWithValue("@name", dev.Name);
        cmd.Parameters.AddWithValue("@commits", dev.Commits);
        cmd.ExecuteNonQuery();
    }

    /// <summary>Обновить данные разработчика</summary>
    public void UpdateDeveloper(Developer dev)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
        UPDATE dev
        SET dep_id = @depId, dev_name = @name, dev_commits = @commits
        WHERE dev_id = @id";
        cmd.Parameters.AddWithValue("@id", dev.Id);
        cmd.Parameters.AddWithValue("@depId", dev.DepartmentId);
        cmd.Parameters.AddWithValue("@name", dev.Name);
        cmd.Parameters.AddWithValue("@commits", dev.Commits);
        cmd.ExecuteNonQuery();
    }

    /// <summary>Удалить разработчика по Id</summary>
    public void DeleteDeveloper(int id)
    {
        using var conn = new SqliteConnection(_connectionString);
```

```
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = "DELETE FROM dev WHERE dev_id = @id";
        cmd.Parameters.AddWithValue("@id", id);
        cmd.ExecuteNonQuery();
    }

    // ──────────── Выполнение произвольного запроса (для отчётов)
────────────

    /// <summary>
    /// Выполняет SQL-запрос и возвращает имена столбцов и строки
результата.
    /// Используется классом ReportBuilder.
    /// </summary>
    public (string[] columns, List<string[]> rows) ExecuteQuery(string sql)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = sql;
        using var reader = cmd.ExecuteReader();

        // Имена столбцов
        string[] columns = new string[reader.FieldCount];
        for (int i = 0; i < reader.FieldCount; i++)
            columns[i] = reader.GetName(i);

        // Строки данных
        var rows = new List<string[]>();
        while (reader.Read())
        {
            string[] row = new string[reader.FieldCount];
            for (int i = 0; i < reader.FieldCount; i++)
                row[i] = reader.GetValue(i)?.ToString() ?? "";
            rows.Add(row);
        }

        return (columns, rows);
    }

    // ──────────── [ГРУППА Б] Экспорт в CSV ────────────

    /// <summary>Экспорт обеих таблиц в CSV-файлы</summary>
    public void ExportToCsv(string depPath, string devPath)
    {
        // Экспорт отделов
        var depLines = new List<string>();
        depLines.Add("dep_id;dep_name");
        foreach (var dep in GetAllDepartments())
            depLines.Add($"{dep.Id};{dep.Name}");
        File.WriteAllLines(depPath, depLines.ToArray());

        // Экспорт разработчиков
```

```
        var devLines = new List<string>();
        devLines.Add("dev_id;dep_id;dev_name;dev_commits");
        foreach (var dev in GetAllDevelopers())

devLines.Add($"{dev.Id};{dev.DepartmentId};{dev.Name};{dev.Commits}");
        File.WriteAllLines(devPath, devLines.ToArray());
    }

    // ──────────── [ГРУППА Г] Фильтр по отделу ────────────

    /// <summary>Получить разработчиков конкретного отдела</summary>
    public List<Developer> GetDevelopersByDepartment(int depId)
    {
        var result = new List<Developer>();
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
        SELECT dev_id, dep_id, dev_name, dev_commits
        FROM dev WHERE dep_id = @depId ORDER BY dev_name";
        cmd.Parameters.AddWithValue("@depId", depId);
        using var reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            result.Add(new Developer(
                reader.GetInt32(0), reader.GetInt32(1),
                reader.GetString(2), reader.GetInt32(3)));
        }
        return result;
    }
}
```

12.4 Файл ReportBuilder.cs

```
using System.Text;

/// <summary>
/// Построитель отчётов с использованием паттерна Fluent Interface.
/// Промежуточные методы возвращают this (цепочка вызовов).
/// Терминальные методы выполняют запрос и форматируют результат.
///
/// Зависит от DatabaseManager (жёсткая связь через конструктор).
/// </summary>
class ReportBuilder
{
    // Жёсткая зависимость от DatabaseManager
    private DatabaseManager _db;

    // Настройки отчёта (заполняются промежуточными методами)
    private string _sql = "";
    private string _title = "";
    private string[] _headers = Array.Empty<string>();
    private int[] _widths = Array.Empty<int>();
    private bool _numbered = false;   // [ГРУППА A]
    private string _footer = "";      // [ГРУППА B]

    /// <summary>
    /// Конструктор. Принимает DatabaseManager для выполнения запросов.
    /// </summary>
    public ReportBuilder(DatabaseManager db)
    {
        _db = db;
    }

    // ──────────── Промежуточные методы (возвращают this) ────────────

    /// <summary>SQL-запрос для отчёта</summary>
    public ReportBuilder Query(string sql)
    {
        _sql = sql;
        return this;
    }

    /// <summary>Заголовок отчёта</summary>
    public ReportBuilder Title(string title)
    {
        _title = title;
        return this;
    }

    /// <summary>Названия колонок для отображения</summary>
    public ReportBuilder Header(params string[] columns)
    {
        _headers = columns;
        return this;
    }
```

```
/// <summary>Ширина каждой колонки в символах</summary>
public ReportBuilder ColumnWidths(params int[] widths)
{
    _widths = widths;
    return this;
}

/// <summary>[ГРУППА A] Включить нумерацию строк</summary>
public ReportBuilder Numbered()
{
    _numbered = true;
    return this;
}

/// <summary>[ГРУППА B] Добавить итоговую строку</summary>
public ReportBuilder Footer(string label)
{
    _footer = label;
    return this;
}

// ──────────── Терминальные методы ────────────

/// <summary>
/// Выполняет запрос, форматирует результат через StringBuilder,
/// возвращает готовую строку отчёта
/// </summary>
public string Build()
{
    // Выполняем запрос через DatabaseManager
    var (columns, rows) = _db.ExecuteQuery(_sql);
    var sb = new StringBuilder();

    // --- Заголовок отчёта ---
    if (_title.Length > 0)
    {
        sb.AppendLine();
        sb.AppendLine($"=== {_title} ===");
    }

    // --- Определяем названия колонок ---
    // Если Header() не вызван, используем имена из SQL-результата
    string[] displayHeaders = _headers.Length > 0 ? _headers : columns;

    // --- Определяем ширину колонок ---
    // Если ColumnWidths() не вызван, ставим 20 на каждую колонку
    int colCount = displayHeaders.Length;
    int[] widths;
    if (_widths.Length >= colCount)
    {
        widths = _widths;
    }
    else
    {
        widths = new int[colCount];
        for (int i = 0; i < colCount; i++)
```

```
                widths[i] = 20;
        }

        // --- Ширина столбца нумерации ---
        int numWidth = _numbered ? 5 : 0;

        // --- Шапка таблицы ---
        if (_numbered)
            sb.Append("№".PadRight(numWidth));

        for (int i = 0; i < colCount; i++)
            sb.Append(displayHeaders[i].PadRight(widths[i]));
        sb.AppendLine();

        // --- Разделитель ---
        int totalWidth = numWidth;
        for (int i = 0; i < colCount; i++)
            totalWidth += widths[i];
        sb.AppendLine(new string('—', totalWidth));

        // --- Строки данных ---
        for (int r = 0; r < rows.Count; r++)
        {
            if (_numbered)
                sb.Append((r + 1).ToString().PadRight(numWidth));

            for (int c = 0; c < rows[r].Length && c < colCount; c++)
                sb.Append(rows[r][c].PadRight(widths[c]));

            sb.AppendLine();
        }

        // --- Итоговая строка [ГРУППА В] ---
        if (_footer.Length > 0)
        {
            sb.AppendLine(new string('—', totalWidth));
            sb.AppendLine($"{_footer}: {rows.Count}");
        }

        return sb.ToString();
    }

    /// <summary>Выполняет Build() и выводит результат в консоль</summary>
    public void Print()
    {
        Console.Write(Build());
    }

    /// <summary>[ГРУППА Б] Сохраняет отчёт в текстовый файл</summary>
    public void SaveToFile(string path)
    {
        File.WriteAllText(path, Build());
        Console.WriteLine($"Отчёт сохранён в файл: {path}");
    }
}
```

12.5 Основная программа

```
using System.Text;

// ═══════════════════════════════════════════════════════════
//  Точка входа — консольное меню
// ═══════════════════════════════════════════════════════════

Console.OutputEncoding = Encoding.UTF8;
Console.InputEncoding = Encoding.UTF8;

// Пути к файлам
string dbPath = "developers.db";
string depCsv = Path.Combine(AppContext.BaseDirectory, "dep.csv");
string devCsv = Path.Combine(AppContext.BaseDirectory, "dev.csv");

// Создаём менеджер БД и инициализируем данные
var db = new DatabaseManager(dbPath);
db.InitializeDatabase(depCsv, devCsv);

Console.WriteLine();

// Главный цикл меню
string choice;
do
{
    Console.WriteLine("╔═══════════════════════════════════════════╗");
    Console.WriteLine("║      УПРАВЛЕНИЕ РАЗРАБОТЧИКАМИ      ╖");
    Console.WriteLine("╟═══════════════════════════════════════════╢");
    Console.WriteLine("╟  1 — Показать все отделы              ");
    Console.WriteLine("╟  2 — Показать всех разработчиков      ");
    Console.WriteLine("╟  3 — Добавить разработчика            ");
    Console.WriteLine("╟  4 — Редактировать разработчика       ");
    Console.WriteLine("╟  5 — Удалить разработчика             ");
    Console.WriteLine("╟  6 — Отчёты                           ");
    Console.WriteLine("╟  7 — Фильтр по отделу   [ГРУППА Г]  ");
    Console.WriteLine("╟  8 — Экспорт в CSV      [ГРУППА Б]  ");
    Console.WriteLine("╟  0 — Выход                            ");
    Console.WriteLine("╚═══════════════════════════════════════╝");
    Console.Write("Ваш выбор: ");

    choice = Console.ReadLine()?.Trim() ?? "";
    Console.WriteLine();

    switch (choice)
    {
        case "1": ShowDepartments(db); break;
        case "2": ShowDevelopers(db); break;
        case "3": AddDeveloper(db); break;
        case "4": EditDeveloper(db); break;
        case "5": DeleteDeveloper(db); break;
        case "6": ReportsMenu(db); break;
        case "7": FilterByDepartment(db); break;  // [ГРУППА Г]
        case "8": ExportCsv(db); break;            // [ГРУППА Б]
        case "0": Console.WriteLine("До свидания!"); break;
```

```
        default: Console.WriteLine("Неверный пункт меню."); break;
    }

    Console.WriteLine();
}
while (choice != "0");

// ═══════════════════════════════════════════════════════════
//  Функции пунктов меню
// ═══════════════════════════════════════════════════════════

static void ShowDepartments(DatabaseManager db)
{
    Console.WriteLine("--- Все отделы ---");
    var departments = db.GetAllDepartments();
    foreach (var dep in departments)
        Console.WriteLine("  " + dep);
    Console.WriteLine($"Итого: {departments.Count}");
}

static void ShowDevelopers(DatabaseManager db)
{
    Console.WriteLine("--- Все разработчики ---");
    var developers = db.GetAllDevelopers();
    foreach (var dev in developers)
        Console.WriteLine("  " + dev);
    Console.WriteLine($"Итого: {developers.Count}");
}

static void AddDeveloper(DatabaseManager db)
{
    Console.WriteLine("--- Добавление разработчика ---");

    // Показываем отделы, чтобы пользователь выбрал
    Console.WriteLine("Доступные отделы:");
    var departments = db.GetAllDepartments();
    foreach (var dep in departments)
        Console.WriteLine("  " + dep);

    // Запрос данных у пользователя
    Console.Write("ID отдела: ");
    if (!int.TryParse(Console.ReadLine(), out int depId))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    Console.Write("Имя разработчика: ");
    string name = Console.ReadLine()?.Trim() ?? "";
    if (name.Length == 0)
    {
        Console.WriteLine("Ошибка: имя не может быть пустым.");
        return;
    }

    Console.Write("Количество коммитов: ");
```

```
    if (!int.TryParse(Console.ReadLine(), out int commits))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    try
    {
        var dev = new Developer(0, depId, name, commits);
        db.AddDeveloper(dev);
        Console.WriteLine("Разработчик добавлен.");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Ошибка: {ex.Message}");
    }
}

static void EditDeveloper(DatabaseManager db)
{
    Console.WriteLine("--- Редактирование разработчика ---");
    Console.Write("Введите ID разработчика: ");

    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var dev = db.GetDeveloperById(id);
    if (dev == null)
    {
        Console.WriteLine($"Разработчик с ID={id} не найден.");
        return;
    }

    Console.WriteLine($"Текущие данные: {dev}");
    Console.WriteLine("(нажмите Enter, чтобы оставить значение без
изменений)");

    // Имя
    Console.Write($"Имя [{dev.Name}]: ");
    string input = Console.ReadLine()?.Trim() ?? "";
    if (input.Length > 0)
        dev.Name = input;

    // Отдел
    Console.Write($"ID отдела [{dev.DepartmentId}]: ");
    input = Console.ReadLine()?.Trim() ?? "";
    if (input.Length > 0 && int.TryParse(input, out int newDepId))
        dev.DepartmentId = newDepId;

    // Коммиты
    Console.Write($"Коммиты [{dev.Commits}]: ");
    input = Console.ReadLine()?.Trim() ?? "";
    if (input.Length > 0 && int.TryParse(input, out int newCommits))
```

```
    {
        try
        {
            dev.Commits = newCommits; // валидация в set-аксессоре
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine($"Ошибка: {ex.Message}");
            return;
        }
    }

    db.UpdateDeveloper(dev);
    Console.WriteLine("Данные обновлены.");
}

static void DeleteDeveloper(DatabaseManager db)
{
    Console.WriteLine("--- Удаление разработчика ---");
    Console.Write("Введите ID разработчика: ");

    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var dev = db.GetDeveloperById(id);
    if (dev == null)
    {
        Console.WriteLine($"Разработчик с ID={id} не найден.");
        return;
    }

    Console.Write($"Удалить «{dev.Name}»? (да/нет): ");
    string confirm = Console.ReadLine()?.Trim().ToLower() ?? "";
    if (confirm == "да")
    {
        db.DeleteDeveloper(id);
        Console.WriteLine("Разработчик удалён.");
    }
    else
    {
        Console.WriteLine("Удаление отменено.");
    }
}

// ============================================================
//  Подменю отчётов
// ============================================================

static void ReportsMenu(DatabaseManager db)
{
    string choice;
    do
    {
```

```
        Console.WriteLine("--- Отчёты ---");
        Console.WriteLine("  1 — Разработчики по отделам");
        Console.WriteLine("  2 — Количество разработчиков в отделах");
        Console.WriteLine("  3 — Среднее коммитов по отделам");
        Console.WriteLine("  0 — Назад");
        Console.Write("Ваш выбор: ");

        choice = Console.ReadLine()?.Trim() ?? "";

        switch (choice)
        {
            case "1": Report1_DevelopersWithDepts(db); break;
            case "2": Report2_CountByDept(db); break;
            case "3": Report3_AvgCommitsByDept(db); break;
            case "0": break;
            default: Console.WriteLine("Неверный пункт."); break;
        }

        Console.WriteLine();
    }
    while (choice != "0");
}

// ──────── Отчёт 1: Разработчики с названиями отделов (JOIN) ────────

static void Report1_DevelopersWithDepts(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"SELECT d.dev_name, dep.dep_name, d.dev_commits
                 FROM dev d
                 JOIN dep ON d.dep_id = dep.dep_id
                 ORDER BY d.dev_name")
        .Title("Разработчики по отделам")
        .Header("Имя", "Отдел", "Коммиты")
        .ColumnWidths(20, 15, 10)
        .Numbered()              // [ГРУППА A] нумерация строк
        .Footer("Всего записей") // [ГРУППА B] итоговая строка
        .Print();
}

// ──────── Отчёт 2: Количество разработчиков по отделам (GROUP BY + COUNT)
────

static void Report2_CountByDept(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"SELECT dep.dep_name, COUNT(*) AS cnt
                 FROM dev
                 JOIN dep ON dev.dep_id = dep.dep_id
                 GROUP BY dep.dep_name
                 ORDER BY dep.dep_name")
        .Title("Количество разработчиков по отделам")
        .Header("Отдел", "Кол-во")
        .ColumnWidths(20, 10)
        .Print();
}
```

```
// ──────── Отчёт 3: Среднее коммитов по отделам (GROUP BY + AVG) ────────

static void Report3_AvgCommitsByDept(DatabaseManager db)
{
    // [ГРУППА Б] Пример сохранения отчёта в файл:
    // Замените .Print() на .SaveToFile("report3.txt")

    new ReportBuilder(db)
        .Query(@"SELECT dep.dep_name,
                        ROUND(AVG(dev.dev_commits), 1) AS avg_commits
                 FROM dev
                 JOIN dep ON dev.dep_id = dep.dep_id
                 GROUP BY dep.dep_name
                 ORDER BY avg_commits DESC")
        .Title("Среднее количество коммитов по отделам")
        .Header("Отдел", "Среднее коммитов")
        .ColumnWidths(20, 20)
        .Print();
}

// ═══════════════════════════════════════════════════════════════
//  [ГРУППА Г] Фильтр по отделу
// ═══════════════════════════════════════════════════════════════

static void FilterByDepartment(DatabaseManager db)
{
    Console.WriteLine("--- Фильтр по отделу ---");
    Console.WriteLine("Доступные отделы:");
    var departments = db.GetAllDepartments();
    foreach (var dep in departments)
        Console.WriteLine("  " + dep);

    Console.Write("Введите ID отдела: ");
    if (!int.TryParse(Console.ReadLine(), out int depId))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var developers = db.GetDevelopersByDepartment(depId);
    if (developers.Count == 0)
    {
        Console.WriteLine("В этом отделе нет разработчиков.");
        return;
    }

    Console.WriteLine($"\nРазработчики отдела #{depId}:");
    foreach (var dev in developers)
        Console.WriteLine("  " + dev);
    Console.WriteLine($"Итого: {developers.Count}");
}

// ═══════════════════════════════════════════════════════════════
//  [ГРУППА Б] Экспорт в CSV
// ═══════════════════════════════════════════════════════════════
```

```
static void ExportCsv(DatabaseManager db)
{
    string depPath = Path.Combine(AppContext.BaseDirectory,
"dep_export.csv");
    string devPath = Path.Combine(AppContext.BaseDirectory,
"dev_export.csv");

    db.ExportToCsv(depPath, devPath);
    Console.WriteLine($"Отделы экспортированы в: {depPath}");
    Console.WriteLine($"Разработчики экспортированы в: {devPath}");
}
```